

Lemmatization & BM25/tsvector Keyword Search

What it is

This concept is really three small ideas stacked together. Let's take them one at a time.

Lemmatization means reducing a word down to its dictionary ("base") form. "manufacturing," "manufactures," and "manufactured" are three different words to a computer doing exact text matching, but they all mean the same underlying idea — the base form is "manufacture." Lemmatization is the process of automatically figuring that out and rewriting each word as its base form. This matters for search: if a user types "manufacture" into a search box, they almost certainly also want results that say "manufacturing" or "manufactured" — but a naive keyword search that only checks for the exact characters typed would miss those.

BM25 is a decades-old, extremely well-tested algorithm for scoring how relevant a piece of text is to a search query. It's used by Elasticsearch, Postgres's built-in full-text search, and many other search systems. In plain terms, it looks at: how often does the search term appear in this document (more is usually better, up to a point), how long is the document overall (a term appearing 3 times in a 20-word blurb is more significant than 3 times in a 2,000-word essay), and how rare is this term across *all* documents (a term that appears in almost everything, like "the," tells you little; a term that appears in only a few documents, like "hydraulic," is a strong signal). We do not write any of that scoring math ourselves in this project — Postgres's own full-text search machinery does it for us. What we *do* write is the code that prepares the text Postgres will score.

tsvector is Postgres's built-in data type for full-text search. You can think of it as a compact, pre-processed "search index" for a single row of data — Postgres builds one for each row's searchable text, and later, when a search query comes in, it can check the query against many rows' tsvectors very quickly instead of re-reading and re-scanning every row's full raw text from scratch every time.

Real-life analogy

Think of a tsvector like the index at the back of a textbook, or a library's old card catalog.

When a book's index gets built, nobody sits down and copies out every single sentence of the book. Instead, an editor goes through and pulls out the *meaningful* words — "photosynthesis," "mitochondria," "enzyme" — and for each one, writes down the page numbers where it appears. Small filler words like "the," "and," "of" don't get their own index entries; they're everywhere and they'd make the index useless clutter. That back-of-book index is what lets you flip straight to page 214 for "mitochondria" instead of reading the entire book cover to cover every time you have a question. A tsvector is exactly that idea, but for one row in a database table instead of one book: a short list of the meaningful words in that row's text, ready for fast lookup.

Now here's the part that connects to lemmatization. Imagine the same textbook uses the phrase "enzymes" on page 10, "enzyme" on page 55, and "enzymatic reaction" on page 90. A *good* index groups all of those under one entry — "enzyme" — with all three page numbers listed together, because a reader looking up "enzyme" obviously wants all three pages, not just the one where that exact spelling appears. A *sloppy* index would

create three separate, disconnected entries, and a reader would only find whichever one exact-matched their search. Lemmatization is what makes our "index" (the tsvector) smart enough to group "manufacture," "manufacturing," and "manufactured" together under one entry, the same way a good book editor groups "enzyme" and "enzymes" together.

Why we used it here

This implements **R9 (partial, BM25 path)** — the requirement that the system supports classic keyword search alongside vector/semantic search, fused later via RRF (see `.claude/rules/bilingual-nlp.md` and `SPEC.md`). The specific file is `bilingual_etl/nlp/lemmatizer.py`.

The tricky part of R9 in this project is that our data is **bilingual** — Hungarian and English — and Hungarian is a heavily inflected language where a single root word can appear in dozens of grammatical forms. Postgres does ship built-in language configurations for `to_tsvector()`, including a genuinely strong one for English (`to_tsvector('english', ...)`), which does its own stemming automatically. But there is no comparably strong built-in Hungarian configuration in stock Postgres. If we relied on Postgres alone, English search quality would be fine and Hungarian search quality would be noticeably weaker.

The fix: we do the language-aware lemmatization ourselves, in Python, *before* the text ever reaches Postgres — using `huspacy`, a proper Hungarian NLP model trained specifically for Hungarian grammar (not naive suffix-chopping), for Hungarian text, and `spacy`'s English model for English text. By the time our text reaches the database, it's already been reduced to base forms correctly, for whichever language it's actually in. We then hand that pre-lemmatized text to `to_tsvector('simple', ...)` — the 'simple' configuration does *zero* further stemming; it just splits text into tokens and packages them into the tsvector format. That's intentional: we don't want Postgres re-processing text a second time with a mismatched or (for Hungarian) missing language config on top of work we already did correctly in Python. Postgres's job here is just packaging, not linguistics.

This module also depends directly on two things from earlier phases of this same project:

- `bilingual_etl/nlp/protected_terms.py` (R8, Phase D) — its `ProtectedTermsMasker` class is imported and used to keep brand names safe during lemmatization (see Code walkthrough).
- The language detection performed earlier in the pipeline (`language_detector.py`, Phase B) — by the time a record reaches this module, its language ("hu" or "en") is already known, and that decision is what routes the text to the correct model.

Code walkthrough

Module docstring and setup

```
"""
Lemmatization for BM25 keyword search (R9, partial)

Lemmatization reduces words to their dictionary base form (e.g. "manufacturing"
-> "manufacture") so a keyword search for one form also matches the others.
```

Hungarian and English need different models -- `huspacy` for Hungarian (`hu_core_news_lg`), `spaCy` for English (`en_core_web_lg`) -- routed by the record's detected language. Never cross-apply a model to the wrong language.

This module also wires in Protected Terms masking (R8): the full pipeline for producing BM25-ready text is `mask` -> `lemmatize` -> `unmask`, in that exact order. Masking first means the lemmatizer never sees (and can't corrupt) brand names; unmasking after restores them verbatim in the final token string.

```
import spacy

from bilingual_etl.nlp.protected_terms import ProtectedTermsMasker

MODEL_NAMES = {
    "hu": "hu_core_news_lg",
    "en": "en_core_web_lg",
}

_model_cache: dict[str, spacy.Language] = {}
```

- `MODEL_NAMES` is the routing table: language code `"hu"` maps to the `huspacy` Hungarian model name, `"en"` maps to the `spaCy` English model name. Note that `huspacy` models are loaded through the same `spacy.load(...)` call as regular `spaCy` models — `huspacy` packages its Hungarian model so it plugs into the standard `spaCy` API, which is why the rest of the file doesn't need any `huspacy`-specific code.
- `_model_cache` is a plain dictionary, declared once at module level (outside any function), that will hold each language's *already-loaded* model. It starts empty.

Loading a model, only once per language

```
def _get_model(language: str) -> spacy.Language:
    if language not in MODEL_NAMES:
        raise ValueError(f"Unsupported language '{language}'. Expected 'hu' or 'en'.")

    if language not in _model_cache:
        _model_cache[language] = spacy.load(MODEL_NAMES[language])
    return _model_cache[language]
```

- The first `if` guards against a typo or an unsupported language code (anything other than `"hu"` or `"en"`) reaching the rest of the function, and fails loudly with a clear error message rather than silently doing the wrong thing.
- The second `if` is the caching logic: `spacy.load(...)` is slow (often a full second or more) and memory-heavy (these language models are hundreds of megabytes, holding statistical data about grammar, vocabulary, and word forms). If every single call to `lemmatize()` reloaded the model from scratch, processing thousands of records would be unusably slow and would repeatedly allocate huge amounts of memory for no reason. Instead, the *first* time a language is requested, the model gets loaded and stored in `_model_cache`. Every subsequent call for that same language just returns the already-loaded model instantly from the dictionary — the expensive work happens at most twice per process (once for `"hu"`, once for `"en"`), no matter how many thousands of records get lemmatized.

Lemmatizing plain text

```
def lemmatize(text: str, language: str) -> str:
    if not text:
        return ""

    nlp = _get_model(language)
    doc = nlp(text)
    lemmas = [
        token.lemma_.lower()
        for token in doc
        if not token.is_punct and not token.is_space and not token.is_stop
    ]
    return " ".join(lemmas)
```

- `if not text: return ""` handles the edge case of empty or missing text up front, avoiding wasted work and avoiding a crash from running a language model on nothing.
- `nlp = _get_model(language)` fetches the correct, already-cached model for this record's language — this is the actual language-routing step. A Hungarian record always gets `hu_core_news_lg`; an English record always gets `en_core_web_lg`. The two are never swapped, because doing so would produce garbage: running the English model's statistical grammar rules on Hungarian text (or vice versa) doesn't raise an error — it just silently produces wrong, meaningless lemmas, since the model has no real knowledge of the other language's word forms.
- `doc = nlp(text)` runs the model over the input text. This single call does tokenization (splitting text into words), part-of-speech tagging, and lemmatization all at once, producing a `doc` object where each `token` inside it carries all of that information.
- The list comprehension is where stopwords and punctuation get filtered out:
- `token.is_punct` is `True` for punctuation tokens (commas, periods, question marks, ...) — these carry no search value and get dropped.
- `token.is_space` is `True` for whitespace tokens — also dropped.
- `token.is_stop` is `True` for **stopwords** — very common, low-information words. In English these are words like "the," "is," "a"; in Hungarian, the model's built-in stopword list covers the equivalents, like "a," "az," and "és" ("and"). These words appear in nearly every sentence, so keeping them in the `tsvector` wouldn't help distinguish one document from another — they'd just be noise.
- `token.lemma_` is the model's computed base form for that token, and `.lower()` normalizes casing so "Manufacture" and "manufacture" become the same search token.
- `" ".join(lemmas)` reassembles the surviving, lemmatized, lowercased words back into a single space-separated string — this is the text that will eventually be handed to Postgres's `to_tsvector('simple', ...)`.

Wiring in Protected Terms masking

```
def lemmatize_for_bm25(text: str, language: str, masker: ProtectedTermsMasker) -> str:
    masked_text, mapping = masker.mask(text)
```

```
lemmatized = lemmatize(masked_text, language)
return masker.unmask(lemmatized, mapping)
```

This is the function the rest of the pipeline actually calls, and it runs exactly three steps in this exact order:

1. `masker.mask(text)` — from the R8 module built in the previous phase (`bilingual_etl/nlp/protected_terms.py`) — replaces every known brand name, product code, or certificate code in `text` with a meaningless placeholder token, and returns a `mapping` remembering which placeholder stood for which original term.
2. `lemmatize(masked_text, language)` — the function above runs on the *masked* text. Since the brand names are already hidden behind placeholders at this point, the lemmatizer never sees them and has no opportunity to mangle them.
3. `masker.unmask(lemmatized, mapping)` — swaps every placeholder back out for its original, exact text.

Concrete example, taken directly from this project's real test suite

(`tests/etl/test_lemmatizer.py::test_real_dataset_brand_piranha`), using two actual brand names from this project's dataset:

```
input: "Piranha pumps are manufactured with ABS components."
output: "Piranha pump manufacture ABS component"
```

Walking through what happened: "Piranha" and "ABS" are both in the protected-terms whitelist, so they were masked, passed through the lemmatizer untouched, and unmasked back to their exact original spelling. Meanwhile "pumps" → "pump" and "manufactured" → "manufacture" are ordinary words, not on the whitelist, so the lemmatizer was free to reduce them to their base form as usual. "are" and "with" disappeared entirely — they're English stopwords, filtered out by the `token.is_stop` check.

A caveat worth being honest about: this project's own rules documentation

(`.claude/rules/bilingual-nlp.md`) uses "Bunting" → "bunt" as the illustrative, worst-case example of what an *unprotected* lemmatizer could theoretically do to a brand name. In practice, when this was actually tested against real spaCy during development, spaCy's English model did **not** reduce "Bunting" to "bunt" — its statistical model recognized enough context to leave it alone. That's genuinely reassuring for this one word, but it doesn't change the decision to mask brand names anyway, and here's why: the whitelist this project protects contains roughly **2,500 real brand names**, product codes, and company names. Even if spaCy happens to handle "Bunting" correctly, there is no guarantee it handles all 2,500 of them correctly, for every grammatical context they might appear in, in both English and Hungarian. Masking isn't a defense against one specific word failing — it's a policy that removes the *entire category* of risk, guaranteeing zero corruption regardless of what any particular lemmatizer does or doesn't get right for any particular brand name. Relying on a lemmatizer "getting lucky" 2,500 times over is not a strategy; masking makes the question irrelevant.

How to explain this in an interview/to a teammate

"We support classic keyword search alongside vector search, and for that we need Postgres's full-text search feature, which relies on a data type called a tsvector — basically a per-row search index, like the index at the back of a book. To make that index actually useful across two languages, we lemmatize the text ourselves in

Python before it reaches Postgres: reducing words like 'manufacturing' and 'manufactured' down to a shared base form, 'manufacture,' so a search for one form finds documents using any of them. We route Hungarian text through huspacy and English text through spaCy, because Postgres doesn't have a strong built-in Hungarian stemmer, and we never mix the two models up. Before any of that lemmatization runs, we mask brand names and product codes with placeholder tokens, so the lemmatizer can't accidentally turn a real brand name into nonsense — and then we unmask them afterward so they come back exactly as they started. It's a small pipeline — mask, lemmatize, unmask — but each step exists to solve a very specific, real failure mode."